

TP n° 1 : Ollama et StreamLit

"Ollama est un outil open-source révolutionnaire qui permet d'exécuter des grands modèles de langage (comme Llama 3, Mistral, ou Phi-3) localement, directement sur votre propre ordinateur. Conçu pour simplifier au maximum le déploiement d'intelligences artificielles, il masque toute la complexité technique en encapsulant les modèles et leurs dépendances dans une interface logicielle légère et ultra-rapide.

Que ce soit via une simple invite de commandes dans le terminal ou à travers son API locale (qui s'intègre parfaitement avec des scripts Python ou des applications Streamlit), Ollama offre aux développeurs et aux passionnés une liberté totale. C'est la solution idéale pour concevoir des applications intelligentes de manière confidentielle, gratuite, et totalement indépendante d'une connexion internet ou de services cloud payants." (description générée par Gemini)

"Streamlit est un framework Python open-source qui permet de transformer des scripts de code en applications web interactives et élégantes en seulement quelques lignes, ce qui en fait le compagnon idéal d'Ollama pour créer des interfaces d'Intelligence Artificielle locales. Grâce à son intégration native avec la bibliothèque Python d'Ollama, Streamlit permet de connecter instantanément vos modèles de langage locaux à des composants web dynamiques (comme des zones de chat, des boutons ou des combobox).

En combinant la légèreté de Streamlit pour l'interface et la puissance d'Ollama pour la génération de texte confidentielle et gratuite, vous pouvez prototyper et déployer des applications d'IA générative (comme des chatbots avec recherche web ou des assistants documentaires) de manière incroyablement rapide, esthétique et entièrement locale." (description générée par Gemini)

Dans cette séance, vous allez déployer plusieurs LLM sur Ollama en mode terminal, puis développer un script Python utilisant StreamLit pour créer une interface Web pour votre ChatBot.

Utilisation de Ollama en mode console

- a) Commencez par télécharger l'exécutable Ollama sur votre machine : `https://ollama.com/download/windows`
- b) Lancez l'exécutable pour l'installer. Si vous êtes sur une des machines de l'école, appelez l'enseignant pour avoir les accès admin
- c) Tester que tout est bien installé en ouvrant un terminal et en tapant `ollama`
- d) Une fois que la vérification est effectuée, coupez le processus
- e) Télécharger un premier modèle à tester : `mistral`. Pour cela, tapez dans la console `ollama pull mistral`
- f) Tapez `ollama list`, vous devriez voir apparaître le LLM que vous venez de télécharger
- g) Exécutez-le avec la commande `ollama run mistral`
- h) Intéragissez avec le chat bot pour :
 - Trouver quelle est la date de ses données les plus récentes
 - L'interroger sur l'ESEO
- i) Terminez votre conversation avec la commande `/bye`

Le chatbot avec lequel vous venez d'interagir est performant, mais ces données ne sont ni à jour ni suffisamment précises. C'est pourquoi vous allez mettre en place dans les séances suivantes un RAG et des requêtes Internet pour l'alimenter en données. Pour l'instant, vous allez créer un script Python qui exploite Ollama, cela permettra d'avoir une surcouche prête à l'emploi

Interface Web StreamLit

- a) Commencez par effectuer les installations via pip de ollama et streamlit
- b) Dans le même dossier, créez les fichiers `app.py` et `LLM.py`. Le premier vous servira à définir l'interface graphique et le second contient l'interface avec Ollama pour exécuter le ChatBot
- c) Tout en haut du fichier, ajoutez les import suivant :
 - `from LLM import *`
 - `import streamlit as st`
- d) Définissez votre fonction **main** et ajoutez son appel dans le point d'entrée du script

- e) Dans la fonction **main** définissez simplement le nom de votre StreamLit (utilisez la fonction `st.title`)
- f) Lancez votre script Python. Attention, StreamLit s'exécute un peu différemment, il faudra utiliser la commande suivante :

```
python -m streamlit run .\app.py
```

(Python lance StreamLit qui lance app.py)
- g) Une page d'adresse `http://localhost:8501` devrait s'ouvrir dans votre navigateur internet (elle ne contient pour l'instant que le titre que vous avez défini)
- h) Dans votre fonction `main` ajoutez un composant de type `st.chat_input` qui permet d'avoir une zone de texte pour que l'utilisateur envoie des messages. Ce composant prend en paramètre le texte qui est en *placeholder* et renvoie le texte tapé par l'utilisateur (que vous stockerez dans une variable)
- i) Juste après, dans une condition qui vérifie que la variable n'est pas vide, faites un `print`
- j) Sauvegardez votre script, mais ne relancez pas le processus, StreamLit se met à jour à la volée.
- k) Rechargez votre navigateur Internet (ou pas, il est possible qu'il se soit également rechargé tout seul), et tapez des messages dans la nouvelle zone de texte. Ces messages devraient apparaître dans votre terminal
- l) Dans le script `LLM.py` définissez la fonction `session_init` qui teste si `st.session_state` contient déjà un attribut `message` et si ce n'est pas le cas, l'initialise comme une liste vide.
- m) Ajoutez la fonction `get_previous_messages` qui renvoie cette liste de message si elle existe, et une liste vide sinon.
- n) Ajoutez la fonction `query` qui prend en paramètre une chaîne de caractères `user_input` (qui contient le message envoyé par l'utilisateur)
- o) Dans cette fonction, ajoutez le `user_input` dans la liste des messages de la session sous la forme d'un dictionnaire :
 - `"role" : "user"`
 - `"content" : user_input`
- p) De retour dans la fonction `main` de `app.py`, ajoutez l'initialisation de la session juste après la définition du titre
- q) Remplacez le `print` par un appel à la fonction `query`
- r) Ensuite, parcourez les messages de la session et écrivez les dans l'interface graphique. Pour cela, vous utiliserez le code suivant (où `msg` est le nom de la variable contenant le dictionnaire qui représente le message) :

```
with st.chat_message(msg["role"]) :
    st.write(msg["content"])
```
- s) Sauvegardez votre script et retournez sur votre navigateur Web
- t) Tapez du texte dans le champ prévu à cet effet, vos messages devraient apparaître dans le chat
- u) Ajoutez à la fin de la fonction `query` un message écrit par le bot (son rôle est "assistant") dont le contenu est toujours le même (c'est pour l'instant seulement un test)
- v) Sauvegardez votre script et retournez sur votre navigateur Web
- w) Tapez du texte dans le champ prévu à cet effet, vos messages devraient apparaître dans le chat avec le bot qui vous répond toujours la même chose

Le lien avec Ollama

L'objectif de cette partie est de faire en sorte que le bot ne nous réponde pas par un message prédéfini, mais que le LLM stocké dans Ollama soit appelé

- a) Commencez par importer `ollama` dans votre script `LLM.py`
- b) Ouvrez un bloc de contexte (`with ...`) pour le message de l'assistant à l'aide du composant de chat dédié de Streamlit (`st.chat_message`), en lui passant la chaîne "assistant" en paramètre.
- c) À l'intérieur de ce bloc, créez un conteneur vide (`st.empty()`) nommé `response_placeholder`. Il servira à afficher le texte en streaming.
- d) Initialisez une variable textuelle vide nommée `full_response` qui servira à accumuler les morceaux de texte reçus.
- e) Appelez la méthode de chat d'Ollama (`ollama.chat`) en lui fournissant les paramètres suivants :

- `model` : mettez pour l'instant `mistral`.
 - `messages` : Une liste de deux messages : le premier a pour rôle `system` et contient le prompt système (ou préprompt) que vous définirez de manière fixe pour l'instant; le second contient l'input de l'utilisateur
 - `stream` : mettez-le à `True` pour recevoir la réponse de manière progressive.
- f) Stockez le flux de données renvoyé dans une variable `response`.
- g) Créez une boucle `for` pour parcourir chaque morceau (chunk) du flux `response`.
- h) Dans cette boucle, récupérez le contenu textuel du chunk courant et ajoutez-le à la suite de la variable `full_response`.
- i) Mettez à jour le conteneur `response_placeholder` en y écrivant (méthode `write`) la valeur actuelle de `full_response`, suivie d'un curseur visuel (ex : " | ") pour simuler une saisie en direct.
- j) Une fois la boucle terminée (lorsque la réponse est complète), videz ce qu'il reste dans `full_response` dans le conteneur `response_placeholder`.
- k) Sauvegarder la réponse de l'assistant dans l'historique de `st.session_state` avec le rôle assistant.

La barre latérale

Si vous êtes arrivé jusqu'ici, utilisez la documentation de StreamLit <https://docs.streamlit.io/> pour reproduire cette barre latérale :

